AI Cloud Lecture Organizer & Search Platform

Phase 1：需求分析與系統架構規劃書（Phase 1 V5.6 嚴謹工程修正版）

> **版本狀態**：Phase 1 V5.6 完整修訂版（工程缺陷精確修正）

> **最後校正日期**：2026-11-10

> **基準依據**：以《Phase1v5.5》為唯一修改基準，針對 P0/P1 工程問題進行嚴謹收斂與閉環修訂。

> **技術架構鎖定**：React + Vite、Node.js + Express、PostgreSQL 15+（原生 `pg` 連線池，不使用 ORM）、PostgreSQL Full-Text Search（中文 2-gram，不引入外部向量庫）、PDF.js、Render.com 單一 Web Service + Render PostgreSQL。

> **程式碼隔離保證**：截至本版本為止，專案仍處於 Phase 1 需求規劃與系統架構定義階段，工作區目錄未產生任何應用程式原始碼。

---

1. 專案目標與核心需求（Project Vision & Core Requirements）

1.1 核心專案目標

打造一款針對大專院校與各類專業課程的高效學習輔助平台。系統能夠安全匯入、持久化保存並精準解析多種格式的教學講義與課堂素材，透過中文雙字（2-gram）全文檢索技術、個人筆記系統（具備版本控制與草稿保護機制）以及多模型 AI 助理，提供高準確度、具備實體文件精確定位能力的學習體驗。

1.2 邊界與技術限制

1. **純 JavaScript 規範**：全端全面採用標準 ECMAScript（JavaScript, ES Modules），嚴格禁止引入 TypeScript。
2. **無 ORM 規範**：直接使用原生 PostgreSQL 驅動（`pg` 連線池），所有 SQL 查詢必須以參數化語法（Parameterized Queries）撰寫，嚴禁任何動態 SQL 字串拼接。
3. **無外部佇列與快取**：完全依賴 PostgreSQL 15+ 的原生交易隔離與 `FOR UPDATE SKIP LOCKED` 機制實作輕量可靠的背景任務排程佇列，嚴格禁止引入 Redis、BullMQ、RabbitMQ 或 Celery。
4. **無外部向量資料庫**：搜尋引擎採用 PostgreSQL 內建 Full-Text Search 與 GIN 索引，搭配客製化中文 2-gram 分詞，嚴格禁止引入 Pinecone、Milvus、Qdrant 或 pgvector。
5. **單一雲端服務實例**：系統架構專為 Render.com 的單一 Web Service 與 Render PostgreSQL 設計，支援於同一進程環境內同時執行 HTTP API 服務與輕量背景 Worker 排程。

---

2. 系統架構（System Architecture）

```
graph TD
    Client["客戶端瀏覽器 (React + Vite + Vanilla CSS)"] -->|HTTPS / REST API| ExpressApp["Express.js API 伺服器 (Node.js 18+)"]

    subgraph ExpressService ["Render Web Service (單一實例進程)"]
        ExpressApp --> AuthModule["身分驗證模組 (express-session + pg-simple)"]
        ExpressApp --> DocRouter["講義管理路由 (/api/documents)"]
        ExpressApp --> SearchRouter["全文檢索路由 (/api/search)"]
        ExpressApp --> NoteRouter["筆記管理路由 (/api/notes)"]
        ExpressApp --> ChatRouter["AI 對話路由 (/api/chat)"]

        DocRouter --> StorageAdapter["雙模式儲存適配層 (StorageAdapter)"]
        DocRouter --> WorkerPool["背景 Worker 池 (Task Scheduler & Parser Adapter)"]
        WorkerPool --> ReaperCron["逾期終止與租約巡檢 (Reaper Cron, 60s)"]
        WorkerPool --> CleanupWorker["孤兒檔案清理行程 (Storage Cleanup Worker)"]
    end

    subgraph StorageLayer ["持久化儲存層 (Persistent Storage)"]
        StorageAdapter -->|STORAGE_DRIVER=local| LocalDisk["Render Persistent Disk (/var/data/storage)"]
        StorageAdapter -->|STORAGE_DRIVER=s3| CloudS3["S3 相容物件儲存 (Cloudflare R2 / AWS S3)"]
    end

    subgraph CloudDB ["Render PostgreSQL Database (v15+)"]
        PG_Session["系統表: session"]
        PG_Cleanup["系統表: storage_cleanup_jobs"]
        PG_Business["業務表: users, courses, documents, chunks, summaries, notes, chat"]
    end

    ExpressApp -->|pg Pool (Parameterized SQL)| CloudDB
    WorkerPool -->|FOR UPDATE SKIP LOCKED| CloudDB
```

3. 多格式檔案管理與支援矩陣（Multi-format Support Matrix）

本專案支援現代教學中最常見的 8 大檔案格式，所有格式均納入統一的生命週期管理與安全審查規範：

格式分類	副檔名	MIME 類型	最大上限	解析核心與提取策略	定位資訊規範（Source Location）
PDF	`.pdf`	`application/pdf`	15 MB	PDF.js 逐頁提取純文字與排版位置	頁碼 `pageNumber`、文字邊框 `bbox: [x1,y1,x2,y2]`
Word	`.docx`	`application/vnd.openxmlformats-officedocument.wordprocessingml.document`	15 MB	安全解壓 XML 串流提取段落文字	段落序號 `paragraphIndex`、大綱路徑 `headingPath`
Excel	`.xlsx`	`application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`	10 MB	SAX 串流逐列解析 XML，防範記憶體暴增	工作表 `sheetName`、列號 `rowIndex`、儲存格範圍 `cellRange`
PPT	`.pptx`	`application/vnd.openxmlformats-officedocument.presentationml.presentation`	20 MB	解壓投影片 XML 提取標題與內容	投影片編號 `slideNumber`、標題 `slideTitle`
純文字	`.txt`	`text/plain`	10 MB	Readline 串流分行讀取	起始行 `startLine`、結束行 `endLine`、字元偏移
Markdown	`.md`	`text/markdown`	10 MB	Readline 串流讀取，提取標題層級	起始行 `startLine`、結束行 `endLine`、章節標題
CSV	`.csv`	`text/csv`	10 MB	串流解析器（`csv-parse`），雙演算法偵測	列號 `rowIndex`、欄位名稱 `columnName`、欄索引
圖片	`.png`, `.jpg`	`image/png`, `image/jpeg`	10 MB	預留 OCR 擴充介面（Phase 2）	邊界座標 `boundingBox: [x,y,w,h]`、置信度

---

4. 原始檔案持久化、備份、復原與儲存清理（Persistent Storage, Backup, Restoration & Consistency）

4.1 雙模式儲存適配層（Dual-Mode Storage Adapter）

系統設計統一抽象介面 `StorageAdapter`，透過環境變數 `STORAGE_DRIVER=local|s3` 切換：

1. **Local Disk 模式（Render Persistent Disk 掛載）**：

- 掛載至持久化磁區（如 `/var/data/storage`），以 `users/{userId}/{documentId}/{uuid}.ext` 結構嚴格隔離，禁止路徑穿越。

1. **S3 相容 Object Storage 模式（雲端多實例擴展）**：

- 支援 AWS S3 或 Cloudflare R2，透過原生 S3 Client 串流讀寫。

4.2 SHA-256 Checksum 完整性驗證機制

- **上傳存檔**：計算檔案之 SHA-256 雜湊，寫入 `documents.checksum_sha256`。
- **解析前校驗**：任何 Worker 開始解析（含 Retry 與 Reprocess）前，讀取實體檔案並重新計算 SHA-256。若不符，立即標記為 `failed` 並記錄 `FILE_CORRUPTED` 錯誤，拒絕執行。
- **下載核對**：客戶端下載時於 Response Header 提供 `ETag` (SHA-256)，供客戶端校驗。

4.3 備份、復原目標、加密金鑰輪替與災難演練（P1-8 規範）

1. **指標定義（RPO / RTO 規格）**：

- **RPO（Recovery Point Objective，復原點目標）**：**24 小時**（每日自動冷備份）。
- **RTO（Recovery Time Objective，復原時間目標）**：**2 小時**（包含資料庫恢復與檔案目錄鏡像還原）。
- **狀態真實性聲明**：目前為系統設計基準（SPECIFIED），尚未於真實生產環境執行破壞性壓力演練（標記為 `NOT EXECUTED`）。

1. **加密金鑰管理與輪替政策（Key Management & Rotation）**：

- 備份檔案採用 `AES-256-GCM` 演算法加密。
- 金鑰儲存於安全環境變數 `BACKUP_ENCRYPTION_KEY`（或雲端 KMS），嚴禁硬編碼。
- **雙金鑰輪替（Dual-Key Rotation）**：支援最新主要金鑰（Primary Key）與過渡金鑰（Previous Key）。備份檔 Header 記錄 `Key-ID`。系統每 90 天執行金鑰輪替，備份時以 Primary Key 加密，還原時依據 `Key-ID` 自動匹配相應金鑰解密。

1. **備份保留與刪除政策（Retention & Deletion Policy）**：

- 採 GFS（Grandfather-Father-Son）輪替模型：
- 日備份（Daily）：保留 7 天。
- 週備份（Weekly）：保留 4 週（每週日冷存檔）。
- 月備份（Monthly）：保留 12 個月。
- 備份檔設定不可變性（Object Lock / WORM），防止惡意竄改與勒索軟體刪除；超期檔案由排程生命週期規則自動永久刪除。

1. **資料庫與原始檔案的一致性保證（DB & Storage Consistency）**：

- **快照時間點同步（Snapshot Alignment）**：每日凌晨排程先執行資料庫 `pg_dump`，並在資料庫日誌記錄時間戳與最新 LSN。實體檔案備份快照緊隨其後進行時間戳同步。
- **一致性校驗器（Consistency Verifier）**：還原程序結束後，自動執行 `npm run backup:verify` 比對腳本，檢查資料庫所有 `documents.storage_path` 於實體儲存中是否存在且 SHA-256 完全符合。

1. **完整還原步驟（Step-by-Step Restoration Procedure）**：

- **步驟 1：服務隔離**。切換 Express 為維護模式（503 Service Unavailable），停止 API 寫入與 Worker 佇列。
- **步驟 2：金鑰取得與解密**。讀取 `BACKUP_ENCRYPTION_KEY`，呼叫解密腳本解密備份檔：

```
openssl enc -d -aes-256-gcm -in backup.dump.enc -out backup.dump -k $BACKUP_ENCRYPTION_KEY
```

- **步驟 3：資料庫還原**。執行 `pg_restore --clean --if-exists -d $DATABASE_URL backup.dump`，確認退出碼為 0。
- **步驟 4：實體檔案還原**。將儲存鏡像快照同步至目標磁區（`/var/data/storage` 或 S3 Bucket）。
- **步驟 5：雙向一致性校驗**。執行 `npm run backup:verify`，全面抽檢重要講義檔案與關聯筆記。
- **步驟 6：重啟服務與煙霧測試**。啟動服務，調用 `/api/health` 確認服務健全。

1. **還原失敗應變處理**：

- 若金鑰解密失敗或校驗碼錯誤：切換至前一日的次要備份檔（Secondary Daily Backup）進行還原，並發送 P0 安全警報。
- 若實體儲存同步中斷：使用 `rsync -P` 支援斷點續傳。

4.4 持久化孤兒檔案清理表與 Worker 租約機制（P1-1 規範）

為杜絕僅依賴記憶體 Log 造成的清理遺漏，資料庫建立專屬持久化清理表 `storage_cleanup_jobs`，補足 Worker 所有權與租約欄位：

```
-- DDL 見第 13 章：包含 worker_id 與 lease_timeout_at
```

清理任務完整 SQL 生命週期規範（P1-1 規範）：

1. **原子認領 SQL（Claim SQL）：**

```
    UPDATE storage_cleanup_jobs
    SET status = 'processing',
        worker_id = $myWorkerId,
        lease_timeout_at = NOW() + INTERVAL '1 minute',
        retry_count = CASE
            WHEN status = 'failed' OR (status = 'processing' AND lease_timeout_at < NOW())
            THEN retry_count + 1
            ELSE retry_count
        END,
        updated_at = NOW()
    WHERE id = (
        SELECT id FROM storage_cleanup_jobs
        WHERE (status = 'pending')
           OR (status = 'failed' AND retry_count < 3 AND (next_retry_at IS NULL OR next_retry_at <= NOW()))
           OR (status = 'processing' AND lease_timeout_at < NOW() AND retry_count < 3)
        ORDER BY created_at ASC
        FOR UPDATE SKIP LOCKED
        LIMIT 1
    )
    RETURNING id, storage_path, reason, retry_count, worker_id;
```

1. **租約續約 SQL（Lease Renewal SQL）：**

```
    UPDATE storage_cleanup_jobs
    SET lease_timeout_at = NOW() + INTERVAL '1 minute',
        updated_at = NOW()
    WHERE id = $jobId
      AND worker_id = $myWorkerId
      AND status = 'processing'
      AND lease_timeout_at > NOW();
```

1. **成功完成 SQL（Completion SQL）：**

```
    UPDATE storage_cleanup_jobs
    SET status = 'completed',
        worker_id = NULL,
        lease_timeout_at = NULL,
        error_message = NULL,
        updated_at = NOW()
    WHERE id = $jobId
      AND worker_id = $myWorkerId
      AND status = 'processing'
      AND lease_timeout_at > NOW();
```

1. **失敗處理與指數退避 SQL（Failure SQL，含租約檢驗）：**

```
UPDATE storage_cleanup_jobs
SET status = 'failed',
    worker_id = NULL,
    lease_timeout_at = NULL,
    error_message = $errorMessage,
    next_retry_at = NOW() + (LEAST(POWER(2, retry_count) * 5, 60) || ' seconds')::interval,
    updated_at = NOW()
WHERE id = $jobId
  AND worker_id = $myWorkerId
  AND status = 'processing'
  AND lease_timeout_at > NOW();
```

1. **逾期接管與重試上限終止 SQL（Reaper for Max Retries）**：

```
UPDATE storage_cleanup_jobs
SET status = 'failed',
    worker_id = NULL,
    lease_timeout_at = NULL,
    error_message = '清理任務逾期且重試達上限 (3 次)，轉入人工審查',
    updated_at = NOW()
WHERE status = 'processing'
  AND lease_timeout_at < NOW()
  AND retry_count >= 3;
```

---

5. 統一 Parser Adapter 介面與有界記憶體緩衝架構（P1-2 規範）

5.1 檔案輸入句柄（Input Handle）規範

```
/**
 * @typedef {Object} ParserInputHandle
 * @property {string} filePath - 本機暫存實體磁碟路徑 (Spool to Disk)
 * @property {number} fileSize - 檔案位元組大小
 * @property {function(): NodeJS.ReadableStream} getStream - 取得串流讀取介面
 * @property {function(number): Promise<Buffer>} getBuffer - 針對小型檔案在安全上限內取得 Buffer
 */

/**
 * @typedef {Object} ParserContext
 * @property {string} documentId - 文件 UUID
 * @property {string} userId - 使用者 UUID
 * @property {string} filename - 原始檔案名稱
 * @property {AbortSignal} signal - 任務取消信號 (用於中斷長任務)
 */
```

5.2 分批輸出契約（Chunk Batching Generator）

```
/**
 * @typedef {Object} ChunkBatch
 * @property {Array<UnifiedChunk>} chunks - 當前批次分塊清單（預設每批 20~50 塊）
 * @property {boolean} isLastBatch - 是否為最後一批
 */

class BaseParser {
  constructor(parserType, parserVersion) {
    this.parserType = parserType;
    this.parserVersion = parserVersion;
    this.supportedMimeTypes = [];
    this.supportedExtensions = [];
  }

  canParse(mimeType, extension) {
    const ext = extension.startsWith('.') ? extension.slice(1).toLowerCase() : extension.toLowerCase();
    return this.supportedMimeTypes.includes(mimeType) || this.supportedExtensions.includes(ext);
  }

  /**
   * 核心分批抽取生成器
   * @param {ParserInputHandle} inputHandle
   * @param {ParserContext} context
   * @returns {AsyncGenerator<ChunkBatch, UnifiedExtractionMetadata, void>}
   */
  async *parseBatches(inputHandle, context) {
    throw new Error('parseBatches() must be implemented by concrete parser');
  }
}
```

5.3 各格式記憶體行為分類與有界預算策略（P1-2 規範）

系統明確區分四類解析模式，**不宣稱未經驗證的絕對有界記憶體**，針對不同格式採取客觀工程防禦：

1. **真串流解析（True Streaming Parsing）**：

- **格式**：`CSV`、`TXT`、`MD`。
- **實作方式**：透過 Node.js `fs.createReadStream` 結合 `csv-parse` Transform Stream 或 `readline` 介面，按行串流讀取，完全不載入全檔。記憶體消耗恆定於緩衝區大小（≤ 10 MB）。

1. **分批/分頁解析（Batched Page-by-Page Parsing with Cleanup）**：

- **格式**：`PDF`。
- **實作方式**：PDF.js 先天需讀取交叉引用表（xref），無法純線性串流。本系統採取「磁碟暫存檔載入 + 逐頁分批處理 + 主動垃圾回收」：
- 使用暫存檔路徑載入文件，限制最大頁數 500 頁。
- 逐頁提取純文字與座標，每處理 5 頁強制呼叫 `page.cleanup()` 釋放內部渲染快取。
- 單頁記憶體限制 50MB，逾時 30 秒熔斷。

1. **整檔載入記憶體與 DOM 展開限制（In-Memory DOM Loading with Hard Caps）**：

- **格式**：`DOCX`、`PPTX`。
- **記憶體膨脹風險**：基於 Office OpenXML 架構，標準函式庫需將內部 XML 解壓並構建 DOM 樹，展開係數達 10~20 倍。
- **防護策略**：實施嚴格檔案大小上限（DOCX: 15MB, PPTX: 20MB），解壓前檢查 ZIP Header 總展開體積不得超過 100MB。Worker 實例分配 256MB 堆疊上限，解析完成立即解構釋放記憶體。

1. **暫存檔處理（Temp File Spooling）**：

- 所有上傳檔案先串流寫入本機暫存檔（Spool to Disk），禁止在 Express 記憶體中保留原始 Buffer。
- 解析完成或失敗時，在 `finally` 區塊無條件刪除暫存檔。

---

6. 正式 Source Location JSON Schema 與跨欄位驗證（P1-6 規範）

6.1 座標原點、單位與數值慣例（Coordinate Conventions）

- **PDF 座標慣例**：
- 原點 $(0, 0)$ 為**頁面左上角（Top-Left）**，X 軸向右遞增，Y 軸向下遞增（由 PDF.js 原生左下角系統精確轉換）。
- 數值表示為 `[x1, y1, x2, y2]`，點數單位（1/72 英吋）或正規化浮點數（`0.0 ~ 1.0`）。
- **跨欄位約束**：強制驗證 $x2 \ge x1$ 且 $y2 \ge y1$。
- **TXT / MD 行號慣例**：
- 行號以 1-based 計數，字元偏移以 0-based 計數。
- **跨欄位約束**：強制驗證 `endLine >= startLine`；若 `endLine === startLine`，則驗證 `endChar >= startChar`。
- **圖片（Image）邊界慣例**：
- 座標採 `[x, y, width, height]`，像素單位。
- **跨欄位約束**：$x \ge 0, y \ge 0, width > 0, height > 0$；且必須符合原圖邊界：$x + width \le imageWidth$ 且 $y + height \le imageHeight$。
- **音訊 / 影片（Audio / Video）時間慣例**：
- 時間格式為 `HH:MM:SS.mmm` 或秒數浮點數。
- **跨欄位約束**：$0 \le startTime \le endTime \le totalDuration$。

6.2 跨欄位邊界校驗實作函式（P1-6 規範）

```
function validateSourceLocation(location, mediaMeta = {}) {
  if (!location || typeof location !== 'object') throw new Error('LOCATION_INVALID');

  switch (location.type) {
    case 'pdf': {
      if (location.bbox) {
        const [x1, y1, x2, y2] = location.bbox;
        if (x2 < x1 || y2 < y1) {
          throw new Error('PDF_BBOX_INVALID: x2 must be >= x1 and y2 must be >= y1');
        }
      }
      break;
    }
    case 'txt':
    case 'md': {
      if (location.endLine < location.startLine) {
        throw new Error('TEXT_RANGE_INVALID: endLine must be >= startLine');
      }
      if (location.endLine === location.startLine && location.endChar !== undefined && location.startChar !
= undefined) {
        if (location.endChar < location.startChar) {
          throw new Error('TEXT_CHAR_RANGE_INVALID: endChar must be >= startChar on same line');
        }
      }
      break;
    }
    case 'image': {
      const [x, y, w, h] = location.boundingBox;
      if (x < 0 || y < 0 || w <= 0 || h <= 0) {
        throw new Error('IMAGE_BBOX_INVALID: Dimensions must be positive');
      }
      if (mediaMeta.imageWidth && mediaMeta.imageHeight) {
        if (x + w > mediaMeta.imageWidth || y + h > mediaMeta.imageHeight) {
          throw new Error('IMAGE_BBOX_EXCEEDS_BOUNDS: Bounding box exceeds image dimensions');
        }
      }
      break;
    }
    case 'audio':
    case 'video': {
      if (location.startTimeSeconds !== undefined && location.endTimeSeconds !== undefined) {
        if (location.endTimeSeconds < location.startTimeSeconds || location.startTimeSeconds < 0) {
          throw new Error('MEDIA_TIME_RANGE_INVALID: endTime must be >= startTime >= 0');
        }
        if (mediaMeta.durationSeconds && location.endTimeSeconds > mediaMeta.durationSeconds) {
          throw new Error('MEDIA_TIME_EXCEEDS_DURATION: endTime exceeds media total duration');
        }
      }
      break;
    }
  }
  return true;
}
```

6.3 完整 JSON Schema (Draft-07)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "SourceLocationSchema",
  "type": "object",
  "oneOf": [
    {
      "title": "PDFLocation",
      "type": "object",
      "required": ["type", "pageNumber"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "pdf" },
        "pageNumber": { "type": "integer", "minimum": 1 },
        "bbox": {
          "type": "array",
          "items": { "type": "number", "minimum": 0 },
          "minItems": 4,
          "maxItems": 4
        }
      }
    },
    {
      "title": "DOCXLocation",
      "type": "object",
      "required": ["type", "paragraphIndex", "headingPath"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "docx" },
        "paragraphIndex": { "type": "integer", "minimum": 0 },
        "headingPath": { "type": "string", "minLength": 1 }
      }
    },
    {
      "title": "TXTLocation",
      "type": "object",
      "required": ["type", "startLine", "endLine"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "txt" },
        "startLine": { "type": "integer", "minimum": 1 },
        "endLine": { "type": "integer", "minimum": 1 },
        "startChar": { "type": "integer", "minimum": 0 },
        "endChar": { "type": "integer", "minimum": 0 }
      }
    },
    {
      "title": "MDLocation",
      "type": "object",
      "required": ["type", "startLine", "endLine"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "md" },
        "startLine": { "type": "integer", "minimum": 1 },
        "endLine": { "type": "integer", "minimum": 1 },
        "headingPath": { "type": "string" },
        "sectionLevel": { "type": "integer", "minimum": 1, "maximum": 6 }
      }
    },
    {
```

```
    "title": "XLSXLocation",
    "type": "object",
    "required": ["type", "sheetName", "rowIndex", "cellRange"],
    "additionalProperties": false,
    "properties": {
      "type": { "const": "xlsx" },
      "sheetName": { "type": "string", "minLength": 1 },
      "rowIndex": { "type": "integer", "minimum": 1 },
      "cellRange": { "type": "string", "minLength": 2 },
      "columnName": { "type": "string" }
    }
  },
  {
    "title": "CSVLocation",
    "type": "object",
    "required": ["type", "rowIndex"],
    "additionalProperties": false,
    "properties": {
      "type": { "const": "csv" },
      "rowIndex": { "type": "integer", "minimum": 1 },
      "columnName": { "type": "string" },
      "columnIndex": { "type": "integer", "minimum": 0 }
    }
  },
  {
    "title": "PPTXLocation",
    "type": "object",
    "required": ["type", "slideNumber"],
    "additionalProperties": false,
    "properties": {
      "type": { "const": "pptx" },
      "slideNumber": { "type": "integer", "minimum": 1 },
      "shapeIndex": { "type": "integer", "minimum": 0 },
      "slideTitle": { "type": "string" }
    }
  },
  {
    "title": "ImageLocation",
    "type": "object",
    "required": ["type", "boundingBox"],
    "additionalProperties": false,
    "properties": {
      "type": { "const": "image" },
      "boundingBox": {
        "type": "array",
        "items": { "type": "number", "minimum": 0 },
        "minItems": 4,
        "maxItems": 4
      },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
    }
  },
  {
    "title": "AudioLocation",
    "type": "object",
    "required": ["type", "startTime", "endTime"],
    "additionalProperties": false,
    "properties": {
      "type": { "const": "audio" },
      "startTime": { "type": "string", "pattern": "^\d{2}:\d{2}:\d{2}(\.\d+)?$" },
      "endTime": { "type": "string", "pattern": "^\d{2}:\d{2}:\d{2}(\.\d+)?$" },
      "speaker": { "type": "string" }
    }
```

```
    },
    {
      "title": "VideoLocation",
      "type": "object",
      "required": ["type", "startTime", "endTime"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "video" },
        "startTime": { "type": "string", "pattern": "^\d{2}:\d{2}:\d{2}(\.\d+)?$" },
        "endTime": { "type": "string", "pattern": "^\d{2}:\d{2}:\d{2}(\.\d+)?$" },
        "subtitleIndex": { "type": "integer", "minimum": 0 }
      }
    }
  ]
}
```

---

7. 試算表（XLSX / CSV）深度規格與偵測演算法（P1-3, P1-4 規範）

7.1 全域檔案與解析資源限制清單

格式分類	最大上傳大小	最大解壓縮大小	壓縮比上限	結構邊界上限	記憶體預算	執行逾時
PDF	15 MB	不適用	不適用	上限 500 頁	256 MB	60 秒
DOCX	15 MB	100 MB	20 : 1	上限 1,000 個 XML 條目	256 MB	60 秒
XLSX	10 MB	50 MB	20 : 1	單表 20,000 列 / 100 欄，上限 50 萬格	256 MB	60 秒
PPTX	20 MB	100 MB	20 : 1	上限 200 頁投影片	256 MB	60 秒
TXT / MD	10 MB	不適用	不適用	上限 50,000 行文字	128 MB	30 秒
CSV	10 MB	不適用	不適用	上限 50,000 列 (串流解析)	128 MB	30 秒

7.2 XLSX 大型 Workbook 記憶體膨脹剖析與 SAX 串流替代策略（P1-3 規範）

- **記憶體膨脹客觀剖析**：

若在 Node.js 中使用 SheetJS（`xlsx`）的 `XLSX.readFile`，底層會將工作表 XML 與 Shared Strings 全部構建為 JavaScript 物件樹。一個 10MB 的壓縮 XLSX 檔案，解壓後 XML 在 V8 堆疊中會膨脹成數十萬個儲存格實例，瞬間耗費 **150MB ~ 300MB** 記憶體。**因此，若全檔載入後才檢查列數，無法防範讀取初期的記憶體暴衝與 OOM 崩潰。**

- **嚴謹處理架構與替代策略**：

1. **SAX 事件驅動串流（SAX-Based Streaming）**：

- 放棄整檔 DOM 解析方式，改採事件驅動的 XML SAX 串流解析器（如 `exceljs` Streaming Workbook Reader 或解開 ZIP 後以 SAX 串流讀取 `xl/worksheets/sheet*.xml`）。
- 逐列（Row-by-Row）提取文字與座標，每累積 50 列即封裝為一個 Chunk 輸出，不保留歷史列物件。

1. **硬性限制與熔斷機制（Circuit Breaker）**：

- XLSX 壓縮檔嚴格限制最大 10MB，ZIP 展開限制 50MB。
- 解析串流累計讀取超過 20,000 列或整體儲存格超過 50 萬格時，立即主動中斷串流並記錄警告，停止後續處理，防止記憶體無上限膨脹。

1. **公式與巨集**：只讀取快取值（`cell.v`），不重新計算公式，嚴格忽略所有 VBA 巨集代碼。

7.3 CSV 多階段強健偵測與使用者顯式解析選項（P1-4 規範）

只讀取前 4KB 或前 10 行無法完全保證偵測正確（例如前幾行為短英數標題，後續才有長中文或多行引號內容）。本系統升級為多階段偵測與使用者手動覆寫機制：

1. **多階段探測與編碼判定順序**：

- **第 1 階段（BOM 判定，100% 精確）**：檢驗前 4KB。若命中 UTF-8 BOM（`0xEF 0xBB 0xBF`）、UTF-16LE（`0xFF 0xFE`）或 UTF-16BE（`0xFE 0xFF`），直接確認編碼。
- **第 2 階段（擴大取樣與嚴格解碼測試）**：若無 BOM，取樣前 64KB 內容，首先以嚴格 UTF-8 解碼器嘗試解碼。若無任何無效字節序，判定為 UTF-8。
- **第 3 階段（Big5 / CP950 語法比對）**：若 UTF-8 解碼拋出異常，將位元組序列送入 Big5 雙字節編碼校驗器（首位元組 `0x81~0xFE`，次位元組 `0x40~0x7E` 或 `0xA1~0xFE`）。若匹配度 $\ge 95\%$，判定為 Big5。
- **第 4 階段（容錯降級）**：若均無法確定，以 UTF-8 降級解碼，非法位元組替換為 Unicode 替代字元（`�`），並於文件記錄 `metadata.warning = 'ENCODING_DEGRADED'`。

1. **引號感知之分隔符號方差演算法（Delimiter Variance with Quoting）**：

- 提取前 50 行（略過空行），透過遵循 RFC 4180 的正規狀態機過濾雙引號內部的字元。
- 統計候選分隔符（逗號 `,`、分號 `;`、Tab ` `、管道符 `|`）在每行中的出現次數。
- 計算各符號在 50 行中的出現次數變異數（Variance），選取**變異數最小且每行平均次數大於零**者為分隔符。

1. **支援使用者顯式指定解析選項（User-Specified Options）**：

- 文件上傳與 Reupload API 支援可選參數：
- `csv_encoding`（`auto` [預設], `utf-8`, `big5`, `gbk`, `utf-16le`）
- `csv_delimiter`（`auto` [預設], `,`, `;`, ` `, `|`）
- `csv_header_line`（整數，預設 1）
- 若使用者顯式提供，系統直接採用使用者指定選項，跳過自動探測，徹底消除誤判可能。

---

8. Reprocess 版本生命週期與 Final Commit 交易安全（P0-2, P0-3 規範）

8.1 統一 `processing_version`（Fencing Token）生命週期

為保證分散式環境與併發請求下的資料一致性，`processing_version` 具有極其明確的生命週期規則：

1. **Initial Upload**：`processing_version = 1`，狀態設為 `pending`，`worker_id = NULL`，`retry_count = 0`。
2. **Worker Claim（唯一正常處理遞增點！）**：

- 由原子認領 SQL 執行：`processing_version = processing_version + 1`。
- 該值即為本次執行的合法 **Fencing Token**。

1. **Reupload 發起時（主動遞增！）**：

- 使用者發起 Reupload 替換新檔時，API 執行條件更新遞增版本，使舊 Worker 立刻失效（詳見第 10 章）。

1. **Cancel 發起時（主動遞增！）**：

- 使用者主動取消時，API 執行更新遞增版本，狀態轉為 `cancelled`。

1. **Final Commit 時（嚴格校驗）**：

- Worker 提交時強制比對版本號，版本若落後立即回滾，舊 Worker 絕無覆蓋新結果之可能。

8.2 Final Commit 完整 Node.js pg 交易契約與實作（P0-3 規範）

以下為生產級 Node.js 原生 `pg` 連線池交易實作，具備完整的 `BEGIN`、`COMMIT`、`ROLLBACK`、`rowCount` 校驗、異常拋出與連線釋放保證：

```
/**
 * 執行講義解析成果之 Final Commit 交易
 * @param {import('pg').Pool} pool - 資料庫連線池
 * @param {Object} params - 提交參數
 * @returns {Promise<{ success: boolean }>}
 */
async function commitDocumentProcessing(pool, {
  docId,
  userId,
  workerId,
  processingVersion,
  newParserType,
  newParserVersion,
  newChunkingVersion,
  chunks // 結構: Array<{ chunkIndex, content, sourceLocation, searchVectorString }>
}) {
  const client = await pool.connect();
  try {
    // 開啟交易
    await client.query('BEGIN');

    // 1. 鎖定主表並核對 Fencing Token、Worker 所有權、狀態與租約
    const checkSql = `
      SELECT id, processing_version, worker_id, lease_timeout_at, status
      FROM documents
      WHERE id = $1
      FOR UPDATE;
    `;
    const checkRes = await client.query(checkSql, [docId]);
    if (checkRes.rowCount === 0) {
      throw new Error(`DOCUMENT_NOT_FOUND: Document ${docId} does not exist`);
    }

    const doc = checkRes.rows[0];
    if (
      doc.status !== 'processing' ||
      doc.processing_version !== processingVersion ||
      doc.worker_id !== workerId ||
      new Date(doc.lease_timeout_at).getTime() <= Date.now()
    ) {
      throw new Error(`LOST_WORKER_OWNERSHIP: Worker ownership lost, lease expired, or version superseded fo
r doc ${docId}`);
    }

    // 2. 刪除該文件之既有舊 Chunks
    // 注意：在交易提交前，PostgreSQL MVCC 保證外部全文檢索仍只能查到舊 Chunks，絕無空窗期！
    await client.query(`DELETE FROM document_chunks WHERE document_id = $1`, [docId]);

    // 3. 批次插入新 Chunks (任一筆失敗即拋出異常觸發 ROLLBACK)
    const insertChunkSql = `
      INSERT INTO document_chunks (
        document_id, user_id, chunk_index, content, source_location, chunking_version, search_vector
      ) VALUES ($1, $2, $3, $4, $5, $6, to_tsvector('simple', $7));
    `;
    for (const chunk of chunks) {
      await client.query(insertChunkSql, [
        docId,
        userId,
        chunk.chunkIndex,
        chunk.content,
        JSON.stringify(chunk.sourceLocation),
        newChunkingVersion,
```

```
        chunk.searchVectorString
      ]);
    }

    // 4. 更新文件主表版本資訊與狀態 (帶入嚴格的所有權與租約過期校驗)
    const updateDocSql = `
      UPDATE documents
      SET status = 'completed',
          parser_type = $1,
          parser_version = $2,
          chunking_version = $3,
          reprocess_count = reprocess_count + 1,
          error_message = NULL,
          lease_timeout_at = NULL,
          worker_id = NULL,
          updated_at = NOW()
      WHERE id = $4
        AND processing_version = $5
        AND worker_id = $6
        AND status = 'processing'
        AND lease_timeout_at > NOW();
    `;
    const updateRes = await client.query(updateDocSql, [
      newParserType,
      newParserVersion,
      newChunkingVersion,
      docId,
      processingVersion,
      workerId
    ]);

    // 5. 核對影響行數 (必須剛好 1 筆，否則代表發生競態衝突)
    if (updateRes.rowCount !== 1) {
      throw new Error(`CONCURRENT_STATE_CONFLICT: Document ${docId} state was mutated concurrently`);
    }

    // 6. 正式提交交易：MVCC 原子生效，舊 Chunks 銷毀，新 Chunks 即刻對外開放檢索
    await client.query('COMMIT');
    return { success: true };
  } catch (err) {
    // 任何失敗全面回滾
    try {
      await client.query('ROLLBACK');
    } catch (rollbackErr) {
      console.error(`Rollback failed for doc ${docId}:`, rollbackErr);
    }
    throw err;
  } finally {
    // 確保連線無條件釋放回連線池
    client.release();
  }
}
```

- **MVCC 隔離與搜尋保護保證**：
- 在 `COMMIT` 執行前，所有新 INSERT 的 Chunks 對外部唯讀查詢完全不可見；既有舊 Chunks 在外部查詢中依然完好可查。
- 若在中途發生任何外鍵衝突、語法錯誤或 Worker 租約過期，`ROLLBACK` 立即執行，資料庫完整還原至交易開始前狀態。
- **只有完整提交成功後，文件才會進入 `completed`，徹底杜絕半完成資料污染。**

---

9. Worker 狀態機、原子認領、續租、逾期終止與失敗防護（P0-1, P0-2, P0-3 規範）

9.1 統一 `retry_count` 核心定義（P0-3 規範）

- **初次執行（Initial Attempt）**：`retry_count = 0`。
- **重試計數增加**：只要不是初次執行，每一次額外發起的執行均計入重試：
- 一般失敗重試：`retry_count` 遞增。
- 租約逾期被其他 Worker 接管：`retry_count` 遞增。
- **最大重試次數（Max Retries）**：`3` 次。
- **最大總執行次數（Max Executions）**：`1（初次）+ 3（重試）= 4` 次。

9.2 原子性任務認領 SQL

```
UPDATE documents
SET status = 'processing',
    worker_id = $myWorkerId,
    processing_started_at = NOW(),
    lease_timeout_at = NOW() + INTERVAL '3 minutes',
    processing_version = processing_version + 1, -- 唯一正常處理遞增點！
    -- 不論從 failed 重試，還是從租約逾期的 processing 接管，一律將 retry_count 遞增！
    retry_count = CASE
        WHEN status = 'failed' OR (status = 'processing' AND lease_timeout_at < NOW())
        THEN retry_count + 1
        ELSE retry_count
    END,
    updated_at = NOW()
WHERE id = (
    SELECT id FROM documents
    WHERE (status = 'pending')
       OR (status = 'failed' AND retry_count < 3 AND (next_retry_at IS NULL OR next_retry_at <= NOW()))
       OR (status = 'processing' AND lease_timeout_at < NOW() AND retry_count < 3) -- 逾期接管受 retry_count
< 3 約束！
    ORDER BY created_at ASC
    FOR UPDATE SKIP LOCKED
    LIMIT 1
)
RETURNING id, user_id, original_filename, storage_path, file_extension, mime_type, processing_version, worke
r_id, retry_count;
```

9.3 逾期且重試超額任務自動終止機制（Reaper Cron，P0-2 規範）

定時排程（Reaper Cron，每 60 秒）掃描終止達到重試上限的逾期任務：

```
UPDATE documents
SET status = 'failed',
    error_message = '任務執行逾時或崩潰，且已達最大重試次數 (3次)，已自動終止。',
    lease_timeout_at = NULL,
    next_retry_at = NULL,
    worker_id = NULL,
    updated_at = NOW()
WHERE status = 'processing'
  AND lease_timeout_at < NOW()
  AND retry_count >= 3;
```

9.4 租約續約 SQL（Lease Renewal）

```
UPDATE documents
SET lease_timeout_at = NOW() + INTERVAL '3 minutes',
    updated_at = NOW()
WHERE id = $docId
  AND worker_id = $myWorkerId
  AND processing_version = $myProcessingVersion
  AND status = 'processing'
  AND lease_timeout_at > NOW();
```

- 若 `result.rowCount === 0`，代表租約已失效或任務被取代，Worker 立即中止處理並退出。

9.5 任務取消流程（Cancellation Flow）

```
UPDATE documents
SET status = 'cancelled',
    processing_version = processing_version + 1, -- 推進版本廢除舊 Worker 權限
    error_message = '使用者主動取消任務',
    lease_timeout_at = NULL,
    worker_id = NULL,
    updated_at = NOW()
WHERE id = $docId
  AND user_id = $userId
  AND status IN ('pending', 'processing');
```

9.6 失敗處理 SQL 與租約驗證（P0-1 修正）

為杜絕過期 Worker 在崩潰或重試時錯誤覆蓋新 Worker 之狀態，Failure SQL 必須帶有完整的四重驗證：

```
UPDATE documents
SET status = 'failed',
    error_message = $errorMessage,
    lease_timeout_at = NULL,
    worker_id = NULL,
    -- 指數退避：2s, 4s, 8s (加隨機抖動 Jitter 0~1000ms)，上限 30s
    next_retry_at = NOW() + (LEAST(POWER(2, retry_count) * 2, 30) || ' seconds')::interval
                          + (random() * 1000 || ' milliseconds')::interval,
    updated_at = NOW()
WHERE id = $docId
  AND processing_version = $myProcessingVersion
  AND worker_id = $myWorkerId
  AND status = 'processing'
  AND lease_timeout_at > NOW(); -- 關鍵 P0-1 修正：租約已過期者絕對禁止寫入失敗狀態！
```

- **後端校驗**：若 `updateRes.rowCount === 0`，代表該 Worker 的租約已逾期或所有權已被接管，Worker 記錄日誌後直接釋放資源，**不得再發起任何狀態變更**。

---

10. Reupload 替換、Fencing Token 遵守與補償流程（P0-2 規範）

當使用者因講義改版而發起 **Reupload** 時，**嚴禁繞過 Worker Claim、租約或 Fencing Token 機制進行同步解析**，必須完全納入非同步任務佇列與版本控制管理：

10.1 併發控制與 Fencing Token 推進

1. **新實體檔案接收**：

- 伺服器接收新檔案串流，校驗 Magic Bytes、MIME、ZIP 結構與 XXE，計算 SHA-256。
- 寫入隔離儲存路徑：`storage/users/{userId}/{docId}/{newUuid}.{ext}`。

1. **資料庫原子重設交易（更新 Documents 主表）**：

```
UPDATE documents
SET original_filename = $newFilename,
    storage_path = $newStoragePath,
    file_size = $newFileSize,
    mime_type = $newMimeType,
    file_extension = $newExt,
    checksum_sha256 = $newChecksum,
    status = 'pending',                          -- 重設為 pending 待認領
    processing_version = processing_version + 1, -- 關鍵：推進 Fencing Token！
    worker_id = NULL,                            -- 清理舊 Worker 所有權
    lease_timeout_at = NULL,                     -- 清理舊租約
    processing_started_at = NULL,                -- 重設開始時間
    next_retry_at = NULL,                        -- 重設退避時間
    retry_count = 0,                             -- 新檔案重設重試次數
    error_message = NULL,
    updated_at = NOW()
WHERE id = $docId
  AND user_id = $userId
  AND status IN ('pending', 'processing', 'completed', 'failed')
RETURNING id, storage_path AS old_storage_path, processing_version;
```

10.2 非同步認領與舊 Worker 失效機制

- **舊 Worker 阻斷**：

若在 Reupload 發生前已有舊 Worker 正在處理舊檔案，其持有的為舊的 `processing_version`。當舊 Worker 嘗試執行續租（9.4）、提交（8.2）或標記失敗（9.6）時，SQL 中的條件：

`WHERE id = $docId AND processing_version = $oldVersion`

受影響筆數必然為 0（`rowCount === 0`）！舊 Worker 立即被阻擋並觸發回滾退出，絕無可能覆蓋新狀態。

- **標準認領**：

重設為 `pending` 之後，由背景 Worker 池透過標準的「9.2 原子性任務認領 SQL」重新競逐認領，推進版本並分配合法租約進行解析，完全符合專案架構規範。

10.3 競態防護與資料庫 / 儲存失敗補償

1. **Reupload 與 Reprocess 併發互斥**：

- 兩者均使用行級鎖更新 `documents` 記錄。後到達的請求將等待前一個請求交易完成。
- 後到達的請求讀取到最新的 `processing_version` 並再次推進版本，確保狀態轉換完全序列化。

1. **儲存與資料庫失敗補償機制**：

- **實體檔案寫入成功，但 DB 更新失敗**：

呼叫 `StorageAdapter.delete(newStoragePath)` 立即刪除新上傳之實體檔案；若刪除失敗，登記至 `storage_cleanup_jobs` 進行重試，杜絕孤兒檔案殘留。

- **DB 更新成功**：

原舊檔案路徑 `old_storage_path` 寫入 `storage_cleanup_jobs`（`reason = 'reupload_old_file'`），待新版本解析確認完畢後非同步清理。

---

11. 檔案預覽與下載 API 安全規格（HTTP 206 Partial Content）

11.1 API 端點清單

- `GET /api/documents/:id/download`：原始檔案安全下載。
- `GET /api/documents/:id/preview`：原始檔案在線預覽，支援 HTTP 206 部分內容。

11.2 HTTP 206 Range 請求與預覽安全性實作

PDF.js 等前端閱讀器在預覽大型講義時，依賴 `Range` 標頭進行分段讀取。預覽 API 必須完整支援 HTTP 206 協議：

1. **標頭防護與 CSP 沙箱**：

- `Content-Security-Policy: default-src 'none'; sandbox;`（禁止預覽環境執行任何 JavaScript 腳本）。
- `X-Content-Type-Options: nosniff`。

1. **範圍請求處理規格**：

```
function handlePreviewRequest(req, res, filePath, fileSize, mimeType) {
  res.setHeader('Accept-Ranges', 'bytes');
  res.setHeader('Content-Type', mimeType);
  res.setHeader('Content-Disposition', 'inline');
  res.setHeader('Content-Security-Policy', "default-src 'none'; sandbox;");
  res.setHeader('X-Content-Type-Options', 'nosniff');

  const range = req.headers.range;
  if (!range) {
    res.setHeader('Content-Length', fileSize);
    return fs.createReadStream(filePath).pipe(res);
  }

  // 解析 bytes=start-end
  const parts = range.replace(/bytes=/, "").split("-");
  const start = parseInt(parts[0], 10);
  const end = parts[1] ? parseInt(parts[1], 10) : fileSize - 1;

  // 邊界校驗 (RFC 7233)
  if (isNaN(start) || isNaN(end) || start > end || start >= fileSize) {
    res.setHeader('Content-Range', `bytes */${fileSize}`);
    return res.status(416).send('Requested Range Not Satisfiable');
  }

  const chunkSize = (end - start) + 1;
  res.status(206);
  res.setHeader('Content-Range', `bytes ${start}-${end}/${fileSize}`);
  res.setHeader('Content-Length', chunkSize);

  const stream = fs.createReadStream(filePath, { start, end });
  stream.pipe(res);
}
```

---

12. SSRF 深度防禦、特殊網段封鎖與 Socket 驗證（P1-5 規範）

12.1 Undici Dispatcher 配置與重定向處理

1. **重定向原則**：

- 預設策略（嚴格禁止）：`maxRedirections: 0`, `redirect: 'manual'`。若伺服器回傳 HTTP 3xx，立即拒絕並拋出 `SSRF_REDIRECT_DISALLOWED`。
- 受控重定向策略（若業務需啟用）：最大重定向跳數為 3。**每一跳（Each Hop）目標 URL 必須重新自第 12.2、12.3 節之 DNS 解析與 IP 白名單檢查完整執行一次**，絕不盲目跟隨跳轉。

12.2 全面 IPv4 / IPv6 特殊與非公開網段封鎖清單

DNS 解析（`dns.promises.resolve4/resolve6`）取得之**所有 IP**，均必須通過嚴格檢驗。僅允許標準公網 Global Unicast IP，封鎖所有非公開網段：

- **IPv4 封鎖清單**：
- `0.0.0.0/8`（本網路，RFC 1122）
- `10.0.0.0/8`（私有網路，RFC 1918）
- `100.64.0.0/10`（電信級 NAT / CGNAT，RFC 6598）
- `127.0.0.0/8`（本機環回 Loopback，RFC 1122）
- `169.254.0.0/16`（Link-Local 網段，雲端 Metadata，RFC 3927）
- `172.16.0.0/12`（私有網路，RFC 1918）
- `192.0.0.0/24`（IETF 協定指派，RFC 6890）
- `192.0.2.0/24`（TEST-NET-1 測試網段，RFC 5737）
- `192.88.99.0/24`（6to4 轉送中繼，RFC 3068）
- `192.168.0.0/16`（私有網路，RFC 1918）
- `198.18.0.0/15`（基準測試網段，RFC 2544）
- `198.51.100.0/24`（TEST-NET-2 測試網段，RFC 5737）
- `203.0.113.0/24`（TEST-NET-3 測試網段，RFC 5737）
- `224.0.0.0/4`（多播 Multicast，RFC 5771）
- `240.0.0.0/4`（保留網段，RFC 1112）
- `255.255.255.255/32`（廣播）
- **IPv6 封鎖清單**：
- `::/128`（未指定位址）
- `::1/128`（本機環回 Loopback）
- `::ffff:0:0/96` 與 `::ffff:0:0:0/96`（IPv4-mapped / IPv4-translated IPv6，**後端強制解包還原為 IPv4，重新帶入上述 IPv4 完整封鎖清單比對**）
- `64:ff9b::/96`（IPv4/IPv6 轉換）
- `100::/64`（Discard-only prefix，RFC 6666）
- `2001::/23`（IETF 協定指派）
- `2001:db8::/32`（文件測試網段）
- `2002::/16`（6to4 定址，RFC 3056）
- `fc00::/7`（唯一區域位址 Unique Local Address, ULA）
- `fe80::/10`（Link-Local 位址）
- `ff00::/8`（多播 Multicast）

12.3 DNS 解析驗證、Socket 遠端 IP 校驗與 IP Pinning

為徹底杜絕 DNS Rebinding 攻擊：

1. **DNS 預先解析**：取得目標 Hostname 之所有 A 與 AAAA 記錄，只要任一 IP 命中上述封鎖名單，立即拒絕連線。
2. **IP Pinning（釘選連線）**：

在自訂的 Dispatcher / `http.Agent` 建立連線時，直接指定 TCP 連線至已通過白名單校驗的 IP，並於 TLS SNI 傳入原始 Hostname：

```
function createSafeAgent(pinnedIp, originalHostname) {
  return new http.Agent({
    createConnection: (options, callback) => {
      const socket = net.connect({ host: pinnedIp, port: options.port });
      socket.on('connect', () => {
        // 關鍵：在實際 Socket 連線建立瞬間，雙重校驗遠端 IP
        const actualRemoteIp = socket.remoteAddress;
        if (actualRemoteIp !== pinnedIp || !isGlobalUnicast(actualRemoteIp)) {
          socket.destroy(new Error('SSRF_SOCKET_REMOTE_IP_MISMATCH'));
          return;
        }
        callback(null, socket);
      });
      return socket;
    }
  });
}
```

---

13. 完整資料庫 Schema 與複合外鍵修正

13.1 複合外鍵與 `ON DELETE SET NULL (course_id)`（PostgreSQL 15+ 語法）

為防止單一欄位外鍵造成跨租戶越權，`documents` 與 `notes` 關聯 `courses` 時採用複合外鍵 `(course_id, user_id)`。在 PostgreSQL 15+ 中，當刪除課程時僅將 `course_id` 設為 NULL，保留 `user_id`（因 `user_id` 為 `NOT NULL`）：

```
FOREIGN KEY (course_id, user_id) REFERENCES courses(id, user_id) ON DELETE SET NULL (course_id)
```

13.2 完整資料庫 DDL

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. 系統管理資料表: session (由 connect-pg-simple 自動維護)
CREATE TABLE IF NOT EXISTS "session" (
  "sid" varchar NOT NULL COLLATE "default" PRIMARY KEY,
  "sess" json NOT NULL,
  "expire" timestamp(6) NOT NULL
);
CREATE INDEX IF NOT EXISTS "IDX_session_expire" ON "session" ("expire");

-- 2. 系統管理資料表: storage_cleanup_jobs (持久化孤兒檔案清理表，P1-1 規範)
CREATE TABLE IF NOT EXISTS "storage_cleanup_jobs" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "storage_path" varchar(512) NOT NULL,
  "reason" varchar(100) NOT NULL,
  "status" varchar(30) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'processing', 'completed', 'f
ailed')),
  "worker_id" varchar(64),
  "lease_timeout_at" timestamp with time zone,
  "retry_count" integer NOT NULL DEFAULT 0,
  "next_retry_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "error_message" text,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS "idx_cleanup_jobs_queue" ON "storage_cleanup_jobs"("status", "next_retry_at", "le
ase_timeout_at");

-- 3. 業務資料表: users
CREATE TABLE IF NOT EXISTS "users" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "email" varchar(255) UNIQUE NOT NULL,
  "password_hash" varchar(255) NOT NULL,
  "display_name" varchar(100) NOT NULL,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW()
);

-- 4. 業務資料表: courses (複合 UNIQUE 供多租戶外鍵約束)
CREATE TABLE IF NOT EXISTS "courses" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "name" varchar(255) NOT NULL,
  "code" varchar(50),
  "description" text,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_courses_id_user_id" UNIQUE ("id", "user_id")
);
CREATE INDEX IF NOT EXISTS "idx_courses_user_id" ON "courses"("user_id");

-- 5. 業務資料表: documents (複合外鍵修正 + 租約管理)
CREATE TABLE IF NOT EXISTS "documents" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "course_id" uuid,
  "original_filename" varchar(255) NOT NULL,
  "storage_path" varchar(512) NOT NULL,
  "file_size" integer NOT NULL,
```

```
  "mime_type" varchar(127) NOT NULL,
  "file_extension" varchar(10) NOT NULL,
  "checksum_sha256" varchar(64) NOT NULL,
  "status" varchar(30) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'processing', 'completed', 'f
ailed', 'cancelled')),
  "worker_id" varchar(64),
  "lease_timeout_at" timestamp with time zone,
  "processing_started_at" timestamp with time zone,
  "next_retry_at" timestamp with time zone,
  "retry_count" integer NOT NULL DEFAULT 0,
  "reprocess_count" integer NOT NULL DEFAULT 0,
  "processing_version" integer NOT NULL DEFAULT 1,
  "chunking_version" integer NOT NULL DEFAULT 1,
  "parser_type" varchar(30),
  "parser_version" varchar(30),
  "error_message" text,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_documents_id_user_id" UNIQUE ("id", "user_id"),
  CONSTRAINT "fk_documents_course_user" FOREIGN KEY ("course_id", "user_id") REFERENCES "courses"("id", "use
r_id") ON DELETE SET NULL (course_id)
);
CREATE INDEX IF NOT EXISTS "idx_documents_user_id" ON "documents"("user_id");
CREATE INDEX IF NOT EXISTS "idx_documents_course_id" ON "documents"("course_id");
CREATE INDEX IF NOT EXISTS "idx_documents_claim_queue" ON "documents"("status", "retry_count", "next_retry_a
t");
CREATE INDEX IF NOT EXISTS "idx_documents_reaper" ON "documents"("status", "lease_timeout_at", "retry_coun
t");

-- 6. 業務資料表: document_chunks (含唯一約束與全文檢索 GIN 索引)
CREATE TABLE IF NOT EXISTS "document_chunks" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "document_id" uuid NOT NULL REFERENCES "documents"("id") ON DELETE CASCADE,
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "chunk_index" integer NOT NULL,
  "content" text NOT NULL,
  "source_location" jsonb NOT NULL,
  "chunking_version" integer NOT NULL DEFAULT 1,
  "search_vector" tsvector NOT NULL,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_chunks_doc_chunk_idx" UNIQUE ("document_id", "chunk_index")
);
CREATE INDEX IF NOT EXISTS "idx_chunks_document_id" ON "document_chunks"("document_id");
CREATE INDEX IF NOT EXISTS "idx_chunks_user_id" ON "document_chunks"("user_id");
CREATE INDEX IF NOT EXISTS "idx_chunks_search_vector" ON "document_chunks" USING GIN("search_vector");

-- 7. 業務資料表: document_summaries
CREATE TABLE IF NOT EXISTS "document_summaries" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "document_id" uuid NOT NULL REFERENCES "documents"("id") ON DELETE CASCADE,
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "summary_text" text NOT NULL,
  "key_points" jsonb,
  "model_provider" varchar(50) NOT NULL,
  "model_name" varchar(100) NOT NULL,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS "idx_summaries_document_id" ON "document_summaries"("document_id");

-- 8. 業務資料表: notes (個人筆記系統，含樂觀鎖 version)
CREATE TABLE IF NOT EXISTS "notes" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "course_id" uuid,
  "document_id" uuid,
  "title" varchar(255) NOT NULL,
  "content" text NOT NULL DEFAULT '',
  "version" integer NOT NULL DEFAULT 1,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  CONSTRAINT "fk_notes_course_user" FOREIGN KEY ("course_id", "user_id") REFERENCES "courses"("id", "user_i
d") ON DELETE SET NULL (course_id),
  CONSTRAINT "fk_notes_document_user" FOREIGN KEY ("document_id", "user_id") REFERENCES "documents"("id", "u
ser_id") ON DELETE SET NULL (document_id)
);
CREATE INDEX IF NOT EXISTS "idx_notes_user_id" ON "notes"("user_id");
CREATE INDEX IF NOT EXISTS "idx_notes_course_id" ON "notes"("course_id");
CREATE INDEX IF NOT EXISTS "idx_notes_document_id" ON "notes"("document_id");

-- 9. 業務資料表: chat_sessions
CREATE TABLE IF NOT EXISTS "chat_sessions" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "document_id" uuid REFERENCES "documents"("id") ON DELETE SET NULL,
  "title" varchar(255) NOT NULL,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  "updated_at" timestamp with time zone NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_chat_sessions_id_user_id" UNIQUE ("id", "user_id")
);
CREATE INDEX IF NOT EXISTS "idx_chat_sessions_user_id" ON "chat_sessions"("user_id");

-- 10. 業務資料表: chat_messages
CREATE TABLE IF NOT EXISTS "chat_messages" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "session_id" uuid NOT NULL REFERENCES "chat_sessions"("id") ON DELETE CASCADE,
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "role" varchar(20) NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
  "content" text NOT NULL,
  "source_chunks" jsonb,
  "created_at" timestamp with time zone NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS "idx_chat_messages_session_id" ON "chat_messages"("session_id");
```

---

14. 中文全文搜尋與 Search Vector 重建版本隔離機制（P1-7 規範）

14.1 檢索原理與分詞管道

1. **中文 2-gram 分詞演算法**：

中文連續序列進行雙字切片（如「分散式系統」-> `分散 散式 式系 系統`），英數字元轉為小寫，並以空格連接為 Token 字串。

1. **入庫生成**：呼叫 `to_tsvector('simple', $tokens)` 寫入 `search_vector` 欄位。
2. **檢索策略**：

- **OR 相關性搜尋（主要模式）**：以 `|` 串接 tokens，`ts_rank_cd` 降冪排序。
- **AND 包含搜尋**：以 `&` 串接，僅保證所有 Token 存在。
- **Phrase Search（精確詞組搜尋）**：使用 PostgreSQL `<->` 相鄰運算子與 SQL `AND content LIKE '%' || $phrase || '%'` 二階段雙重比對。

14.2 Search Vector 影子版本隔離與零停機切換管道（P1-7 規範）

當分詞規則升級或 Chunking 演算法版本變更時，必須執行索引重建（`npm run search:reindex`）。為保證重建期間不混用新舊版本向量、搜尋不中斷：

1. **新版本建立（New Version Initialization）**：

宣告新版本號 $V_{new} = V_{current} + 1$。

1. **分批重建（Batched Background Rebuild）**：

背景 Worker 逐批讀取 `document_chunks`（例如每批 100 筆），以新規則重新計算分詞向量，但寫入影子欄位 `search_vector_staging`（或建立標註 `chunking_version = $V_{new}` 的新 Chunks）。

1. **完整性驗證（Integrity Verification）**：

重建完畢後，執行資料校驗查詢：

```
SELECT COUNT(*) FROM document_chunks WHERE search_vector_staging IS NULL;
```

確認未完成筆數為 0，且非空向量覆蓋率達 100%。

1. **原子切換（Atomic Switchover）**：

在單一資料庫交易內原子交換欄位名稱：

```
BEGIN;
ALTER TABLE document_chunks RENAME COLUMN search_vector TO search_vector_old;
ALTER TABLE document_chunks RENAME COLUMN search_vector_staging TO search_vector;
ALTER TABLE document_chunks RENAME COLUMN search_vector_old TO search_vector_staging;
COMMIT;
```

- **切換耗時 < 1 毫秒，零停機，線上檢索瞬間生效！**

1. **舊版本清理（Old Version Cleanup）**：

切換後觀察 24 小時確認系統穩定，執行背景維護刪除暫存影子資料：

```
ALTER TABLE document_chunks DROP COLUMN search_vector_staging;
```

1. **重建失敗回復（Failure Rollback）**：

若重建批次失敗或完整性校驗未過，直接執行：

```
ALTER TABLE document_chunks DROP COLUMN IF EXISTS search_vector_staging;
```

線上搜尋依然完全依賴原有的 `search_vector`，零污染、零風險。

---

15. 個人筆記系統健全性（樂觀鎖定與狀態碼統整）

15.1 樂觀鎖與狀態碼規範

```
// 筆記更新 API 核心防護
const updateSql = `
  UPDATE notes
  SET title = $1, content = $2, course_id = $3, document_id = $4,
      version = version + 1, updated_at = NOW()
  WHERE id = $5 AND user_id = $6 AND version = $7
  RETURNING version, updated_at;
`;
const result = await db.query(updateSql, [title, content, courseId, docId, noteId, userId, clientVersion]);

if (result.rowCount === 1) {
  // 成功：回傳 200 OK 與新版本號
  return res.status(200).json({
    success: true,
    version: result.rows[0].version,
    updated_at: result.rows[0].updated_at
  });
}

// 影響行數為 0：進一步確認原因
const checkRes = await db.query('SELECT version FROM notes WHERE id = $1 AND user_id = $2', [noteId, userI
d]);
if (checkRes.rowCount === 0) {
  return res.status(404).json({ error: 'NOTE_NOT_FOUND' });
} else {
  // 版本不符：回傳 409 Conflict，前端保留未儲存草稿並提示衝突
  return res.status(409).json({
    error: 'OPTIMISTIC_LOCK_CONFLICT',
    serverVersion: checkRes.rows[0].version
  });
}
```

15.2 前端防抖與路由守衛

- **防抖自動儲存**：使用者輸入停頓 1,000ms 後觸發背景儲存。
- **草稿保護**：本地保留最新編輯草稿；若遇 409 衝突或網路中斷，前端以黃色警示標註，禁止覆蓋，並提供「保留本地版本」或「載入雲端版本」雙軌解決機制。
- **路由守衛**：當 `isDirty === true` 時切換頁面，彈出確認對話框，嚴防草稿遺失。

---

16. Render.com 單一服務部署與啟動配置

1. **環境變數配置**：

- `DATABASE_URL`：Render PostgreSQL 連線字串。
- `STORAGE_DRIVER`：`local`（掛載 `/var/data/storage`）或 `s3`。
- `BACKUP_ENCRYPTION_KEY`：AES-256 加密金鑰。
- `SESSION_SECRET`：Session 簽署金鑰。

1. **啟動腳本**：

```
{
  "scripts": {
    "start": "node src/index.js",
    "worker": "node src/worker.js",
    "search:reindex": "node scripts/reindex.js",
    "backup:verify": "node scripts/verify-backup.js"
  }
}
```

1. **健康檢查端點**：

`GET /api/health`：校驗資料庫連線池與儲存磁區可用性，回傳 HTTP 200。

---

17. 完整測試矩陣（56 項具體測試驗收標準）

保留原有 T01~T56，不刪減、不重排。**由於目前處於 Phase 1 需求規劃階段，全數測試狀態嚴格標註為客觀真實的 `NOT EXECUTED`（未執行），拒絕虛假通過標籤**：

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T01	原始檔案持久化保存	使用者登入	上傳一份 PDF 講義	解析完成後，伺服器儲存路徑仍能讀取原始檔案	原始檔案未被刪除，檔案雜湊相符	Phas 3
T02	原始檔案安全下載（RFC 5987）	檔案已上傳	點擊下載按鈕發送請求	回傳檔案串流，Header 附帶正確 RFC 5987 檔名	能下載完整檔案，內容無損	Phas 3
T03	原始檔案在線安全預覽（CSP 沙箱）	PDF 講義已上傳	於 Split View 開啟預覽	瀏覽器安全預覽 PDF，CSP 沙箱生效	正確呈現預覽，無腳本執行風險	Phas 3
T04	伺服器重啟後檔案仍存在	檔案已上傳保存	重啟 Express 伺服器	重新請求 `/download` 與 `/preview`	檔案正常讀取，無 404 錯誤	Phas 3
T05	備份與還原一致性演練	DB 有檔案資料	執行備份與模擬災難還原	恢復 DB 與檔案目錄後資料完整可查	所有檔案與關聯筆記皆可存取	Phas 5
T06	任務 Retry 驗證（原始檔重跑）	檔案狀態為 failed	呼叫 `/api/documents/:id/retry`	重新分派任務，利用既有原始檔重跑解析	狀態成功轉為 completed，Chunks 入庫	Phas 3
T07	任務 Reupload 驗證（替換新檔）	原始檔損毀/遺失	呼叫 `/reupload` 上傳新檔	重新驗證檔案，覆蓋舊檔並重新解析	舊 Chunks 清除，新 Chunks 正確入庫	Phas 3
T08	Reprocess 原子替換（舊資料無損）	檔案 completed	呼叫 `/reprocess` 升級規則	在單一 DB 交易內刪除舊 Chunks 插入新 Chunks	若中途失敗立即回滾，舊資料不遺失	Phas 3
T09	DOCX 深度驗證（缺少核心 XML 攔截）	上傳偽造 DOCX	檢查 ZIP 內部結構	缺少 `word/document.xml` 立即報錯拒絕	偽造檔案被攔截，回傳 400	Phas 3
T10	XLSX 多工作表解析與定位	含 3 個 Sheet 之 Excel	執行上傳解析	3 個工作表文字均擷取，標註 `sheetName`	搜尋可定位各工作表與列號	Phas 3
T11	CSV 多編碼相容（Big5 / UTF-8）	Big5 編碼之 CSV	執行上傳解析	自動嗅探編碼並以正確字集解碼入庫	中文字元無亂碼，列號定位精確	Phas 3
T12	PPTX 投影片定位與標題抽取	上傳含多頁簡報	執行上傳解析	保留 `slideNumber` 與 `slideTitle`	檢索結果精準標註投影片編號	Phas 3
T13	PDF 頁碼定位精確性	上傳 20 頁 PDF	執行上傳解析	保留各頁文字與 `pageNumber`	檢索結果精確對應講義頁碼	Phas 3

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T14	大檔案串流下載（Stream Pipeline）	15MB 講義	呼叫 `/download`	以 Stream Pipeline 傳輸，記憶體無暴增	下載流暢，伺服器記憶體平穩	Phase 3
T15	ZIP Bomb 解壓縮炸彈攔截	上傳高壓縮比 ZIP	檢查未壓縮大小超過 100MB	監控觸發，立即中止連線並刪除暫存	伺服器無崩潰，回傳 400	Phase 3
T16	XXE 外部實體注入防禦	XML 含外部 DTD	執行 Office 解析	禁止載入外部實體，阻斷讀檔	無內部機密檔案洩漏	Phase 3
T17	SSRF 禁止跳轉（3xx 攔截）	Base URL 回傳 302	代理請求該 Base URL	`maxRedirections: 0` 觸發，拒絕跳轉	回傳 SSRF 錯誤，無跳轉連線	Phase 3
T18	SSRF IP Pinning（防 DNS Rebinding）	輸入動態 DNS 域名	解析 A/AAAA 記錄比對黑名單	釘選安全 IP，連線至已驗證 IP	防禦 DNS Rebinding 成功	Phase 3
T19	跨使用者檔案下載越權防護（IDOR）	使用者 A 檔案 ID	使用者 B 發送下載請求	後端查核 `user_id` 不符，回傳 404	成功阻擋越權存取（IDOR）	Phase 3
T20	跨使用者 RAG 知識庫隔離	使用者 A 講義	使用者 B 進行 AI 問答	檢索 SQL 強制 `WHERE user_id = $userId`	使用者 B 絕無可能檢索到 A 的檔案	Phase 3
T21	**Worker 租約逾期重整與接管計數**	Worker 離線逾時	模擬實例逾時超過 3 分鐘	新 Worker 認領接管，原子更新 worker_id、遞增 processing_version 並累計 retry_count；若 retry_count>=3 則由 Reaper 終止轉為 failed	舊 Worker 權限作廢，接管計數與版本精確遞增	Phase 3
T22	舊 Worker 寫入防護（Fencing Token 拒絕）	租約已由新實例接管	舊實例嘗試 COMMIT 寫入	`WHERE worker_id = $old` 匹配 0 筆自動回滾	舊實例未覆蓋新實例資料	Phase 3
T23	筆記樂觀鎖成功（200 OK）	正常編輯筆記	自動儲存送出相同版本號	影響列數為 1，回傳 200 與新版本號	版本號遞增，內容正常儲存	Phase 3
T24	筆記樂觀鎖衝突（409 Conflict）	傳入過期版本號	模擬兩端同時修改	影響列數為 0，後端查核回傳 409 Conflict	本地草稿保留，彈出版本衝突	Phase 3
T25	筆記路由守衛阻擋（草稿永不遺失）	筆記處於 isDirty	嘗試切換至另一篇筆記	觸發路由守衛，等待儲存成功後方切換	未儲存內容絕不丟失	Phase 3
T26	Markdown XSS 防禦（DOMPurify 淨化）	檔案含惡意 `<script>`	渲染預覽與 AI 回覆	`DOMPurify` 淨化所有危險標籤	無惡意腳本執行	Phase 3
T27	API Key 日誌全域脫敏（sk-***）	發送 AI 代理請求	檢查伺服器全域 Log 輸出	標頭與 Body 之 Key 均遮蔽為 `sk-***`	日誌中絕無真實金鑰留存	Phase 3

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T28	CSRF 標頭防禦（X-CSRF-Token）	跨站發起偽造 POST	無攜帶 `X-CSRF-Token`	後端中介層拒絕請求，回傳 403 Forbidden	CSRF 攻擊被成功攔截	Phas 3
T29	帳號刪除級聯清理（DB 與儲存實體）	使用者執行刪除帳號	觸發 ON DELETE CASCADE	業務表全數清理，持久化實體檔案非同步刪除	無任何殘留孤兒檔案與資料	Phas 4
T30	Render 部署健康檢查（/api/health）	部署至 Render Web	呼叫 `/api/health` 端點	回傳 HTTP 200 與 DB 連線池正常狀態	雲端服務正常啟動，路由健全	Phas 5
T31	雙 Worker 同時認領（SKIP LOCKED 互斥）	多實例啟動佇列	兩 Worker 同時認領同一任務	`FOR UPDATE SKIP LOCKED` 互斥生效，各得其一	任務絕不重複指派，無雙重解析	Phas 3
T32	舊 Worker 覆蓋新結果防護	Worker 延遲與重解析	舊 Worker 延遲嘗試寫入 Chunks	Fencing Token 比對不符，寫入被拒絕回滾	舊解析結果絕無覆蓋新結果	Phas 3
T33	Reprocess 併發衝突防護	同一檔案連點兩次 Reprocess	兩請求併發嘗試觸發解析	認領時推進 processing_version，前一請求失效	資料庫內容完全一致，無重複 Chunks	Phas 3
T34	Storage 與 DB 不一致偵測（壞檔 Checksum）	模擬磁碟檔案損毀/遺失	觸發解析前檢查 Checksum	檢驗不符立即拋出異常，標記 FILE_CORRUPTED	壞檔未進入解析，提示重新上傳	Phas 3
T35	備份還原一致性演練與 Checksum 驗證	模擬災難復原演練	還原 DB 與 Storage 檔案目錄	執行校驗腳本比對所有 Checksum	所有檔案與關聯筆記皆可正常存取	Phas 5
T36	IPv4-mapped IPv6 SSRF 變種解包攔截	傳入 `::ffff:127.0.0.1`	發起代理請求觸發驗證	後端解包還原為 IPv4 並命中黑名單	成功攔截變種 IPv6 繞過攻擊	Phas 3
T37	跨使用者課程關聯防護（複合外鍵約束）	使用者 A 建立講義	嘗試將 course_id 設為使用者 B 的課程	PostgreSQL 觸發複合外鍵違反異常	資料庫直接拒絕跨使用者關聯	Phas 2
T38	跨使用者筆記文件關聯防護（複合外鍵約束）	使用者 A 建立筆記	嘗試將 document_id 設為使用者 B 的檔案	PostgreSQL 觸發複合外鍵違反異常	資料庫直接拒絕跨租戶關聯	Phas 2
T39	**Worker 崩潰重試上限邊界測試**	模擬解析連續拋出異常	Worker 依序執行 3 次失敗重試	初次執行為 0，第 3 次重試失敗後 retry_count=3，認領 SQL 不再匹配，Reaper 排除	任務停留在 failed，總執行 4 次後終止，無無限循環	Phas 3
T40	退避期間重啟不提前認領（next_retry_at）	失敗任務進入退避期	重啟 Express 伺服器	認領 SQL 比對 next_retry_at <= NOW()	退避到期前絕不提前認領	Phas 3

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T41	**Reupload 解析失敗與補償保護測試**	正常檔案已完成	上傳損壞新檔發起 Reupload	儲存或解析失敗觸發補償，新暫存檔清除，舊資料無損	舊檔案、舊 Chunks 完好無損，無半完成殘留	Phas 3
T42	TXT / MD 定位 Schema 驗證	上傳純文字與 Markdown	驗證生成的 source_location	通過 JSON Schema 驗證，endLine >= startLine	檢索結果精確標記行號與標題	Phas 3
T43	Parser Stream 大檔處理（記憶體不暴增）	上傳 10MB CSV / TXT	透過 getStream() 串流解析	有界緩衝運算，無 OOM	伺服器記憶體平穩，解析成功	Phas 3
T44	Chunk 唯一性約束防護（拒絕重複索引）	模擬併發寫入同 index	嘗試 INSERT 相同 document_id 與 chunk_index	觸發 uq_chunks_doc_chunk_idx 唯一衝突異常	資料庫層級拒絕重複 Chunk	Phas 2
T45	Lease Expiration 重複崩潰計數驗證	模擬 Worker A 認領崩潰，接管 Worker B 亦崩潰	連續模擬崩潰	接管逾期任務時計數正確遞增，達 3 次停止接管	任務進入 failed，無無限接管循環	Phas 3
T46	Cancelled 後舊 Worker 嘗試更新狀態攔截	任務進行中被使用者取消 (cancelled)	舊 Worker 嘗試發送更新 failed 或 Commit	由於 status !== 'processing'，影響行數為 0 回滾	cancelled 狀態不被覆蓋，無新 Chunk	Phas 3
T47	複合外鍵刪除驗證 (ON DELETE SET NULL (course_id))	建立課程與關聯講義	刪除該課程 (DELETE FROM courses)	講義 course_id 設為 NULL，user_id 完好保留	刪除成功，無 NOT NULL 違規報錯	Phas 2
T48	**Lease 逾期後舊 Worker 提交阻擋測試**	Worker 續租逾期或心跳丟失	嘗試發起 Final Commit 交易提交	Commit SQL 驗證 lease_timeout_at > NOW() 失敗，rowCount=0 回滾	Commit 被拒絕回滾，舊 Worker 無法覆蓋新資料	Phas 3
T49	**Reupload 與 Reprocess 併發互斥測試**	同時發起 Reupload 與 Reprocess	觀察資料庫行級鎖排定與版本更新	Reupload 取得排他鎖，推進 processing_version，舊 Reprocess Worker 失效	資料庫版本嚴密一致，無混合 Chunks	Phas 3
T50	Parser 大檔分批處理測試 (Async Generator)	上傳包含 1 萬列之巨大 CSV 檔案	啟動 Async Generator 分批產生 Chunks	Worker 逐批寫入，記憶體使用率平穩無跳躍	成功完成解析，無 OOM 崩潰	Phas 3
T51	**processing_version 遞增一致性測試**	依序執行上傳、認領、取消、重試	檢查每次資料庫內 processing_version 之數值變化	僅在認領與主動取消/替換時推進版本，生命週期一致	每個 Worker 取得唯一 Fencing Token	Phas 3
T52	**retry_count 邊界測試**	模擬初次執行、失敗重試、逾期接管	檢查每次失敗後 retry_count 數值	初次為 0，第 1 次失敗為 1，第 3 次失敗停止	精確達到最大總執行次數 4 次終止	Phas 3

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T53	**逾期且重試超額任務終止測試 (Reaper Cron)**	任務 retry_count=3 且逾期停在 processing	觸發 Reaper Cron 逾期掃描 SQL	狀態自動轉為 failed，記錄逾期終止錯誤	任務不再停留在 processing	Phas 3
T54	**過期 Worker Failure SQL 寫入拒絕測試**	Worker 租約過期後嘗試寫入失敗狀態	執行 Failure UPDATE SQL	條件 lease_timeout_at > NOW() 匹配 0 筆回滾	過期 Worker 無法更改任務狀態	Phas 3
T55	**Search Vector 重建版本隔離測試**	執行 search:reindex 索引重建腳本	於重建期間進行 FTS 查詢	查詢始終命中有效 Chunks，無索引損壞空窗	索引重建完成，原子交換欄位無縫切換	Phas 3
T56	**Storage Cleanup Worker 崩潰與接管測試**	清理任務執行中模擬 Worker 崩潰	清理排程認領逾期清理任務	逾期任務被正確接管並重新刪除檔案	實體孤兒檔案最終成功刪除	Phas 3

18. Phase1v5.5 → Phase1v5.6 修改紀錄表（P0 / P1 修正對照）

優先級	原本設計 (Phase1v5.5)	修改後設計 (Phase1v5.6)	修改原因與技術依據	修改章節	是否需實測
P0-1	Failure SQL 未校驗 `lease_timeout_at > NOW()`，過期 Worker 恐錯誤更新失敗狀態	**補齊四重校驗**：`processing_version`、`worker_id`、`status='processing'` 及 `lease_timeout_at > NOW()`	徹底阻絕過期 Worker 寫入失敗狀態或覆蓋接管 Worker 之結果	第 9, 17 章 (T54)	是
P0-2	Reupload 流程存在同步解析描述，且未嚴格與 Worker Claim、Fencing Token 閉環	**重新設計 Reupload 非同步流程**：推進版本廢除舊 Worker，重設狀態為 pending，由背景 Worker 標準認領處理	確保 Reupload 絕不繞過 Fencing Token 與 Worker 租約機制	第 10, 17 章 (T41, T49)	是
P0-3	Final Commit 僅以偽代碼描述，缺少生產級交易流程與完整錯誤處理	**提供完整 Node.js 原生 `pg` 連線池交易實作**，包含 `BEGIN` / `COMMIT` / `ROLLBACK`、`rowCount` 檢驗與 `release()`	建立具體可落地的資料庫交易安全契約	第 8 章	是
P1-1	`storage_cleanup_jobs` 缺少 `worker_id` 與 `lease_timeout_at`，無完整任務 SQL	**補足 DDL 欄位與 5 大完整 SQL**（Claim, Renewal, Completion, Failure, Reaper）	使孤兒檔案清理具備與講義解析相同的高可靠租約機制	第 4, 13, 17 章 (T56)	是
P1-2	Parser 記憶體未客觀分類，宣稱有界卻未區分串流與全載入	**明確分類四種記憶體模式**：真串流 (CSV/TXT)、分批釋放 (PDF)、全載入上限 (DOCX/PPTX) 與暫存檔處理	依據客觀工程事實定義記憶體行為，明確標註限制	第 5 章	是
P1-3	XLSX 僅限制工作表格數，未解決 SheetJS 記憶體膨脹風險	**剖析 DOM 記憶體膨脹**，提出 SAX 事件串流替代策略，實施 2 萬列/50 萬格熔斷	防範惡意試算表導致 Worker OOM 崩潰	第 7 章	是
P1-4	CSV 僅讀取前 4KB / 10 行偵測，缺少大檔容錯與手動選項	**實施 64KB 多階段嗅探、RFC 4180 方差演算法**，支援 API 手動指定編碼與分隔符	解決複雜 CSV 之編碼誤判與格式混亂問題	第 7 章	是
P1-5	SSRF 清單缺少特殊網段，且未規定 Socket Remote Address 檢驗	**補齊 2001::/23、2002::/16 等特殊網段**，實作連線建立時 Socket 遠端 IP 雙重驗證	杜絕 DNS Rebinding 與變種私有位址繞過攻擊	第 12 章	是
P1-6	Source Location 缺少具體跨欄位邊界校驗實作函式	**規範座標原點（左上角），提供 `validateSourceLocation` 跨欄位校驗函式**	嚴格驗證 BBox 及行號範圍合法性	第 6, 17 章 (T42)	是
P1-7	Search Vector 重建未定義零停機版本切換步驟	**建立影子欄位分批重建、完整性驗證、原子切換與失敗回復標準流程**	確保索引維護期間線上檢索零中斷、無新舊混用	第 14, 17 章 (T55)	是
P1-8	備份規格缺少金鑰輪替、一致性驗證與災難復原步驟	**補齊 AES-256-GCM 雙金鑰輪替、GFS 保留政策、一致性驗證器與 6 步還原程序**	建立健全的企業級災難復原技術規範	第 4 章	是

19. 最終自我審查清單（強制跨章節一致性檢核）

- [x] **1. Failure SQL 租約驗證（P0-1）**：第 9.6 節 Failure SQL 已明確加入 `AND lease_timeout_at > NOW()`，並同步修正 T54 測試案例。
- [x] **2. Reupload 遵守 Fencing Token（P0-2）**：第 10 章重新規範 Reupload 流程，上傳後更新主表推進 `processing_version` 並設為 `pending`，由背景 Worker 透過標準 Claim 流程認領處理，舊 Worker 提交自動被拒絕（T41, T49 同步更新）。
- [x] **3. Final Commit 完整交易契約（P0-3）**：第 8.2 節提供完整可運行的 Node.js `pg` 交易實作，含 `BEGIN`、`COMMIT`、`ROLLBACK`、`rowCount` 驗證、舊 Chunks MVCC 保護與 `finally client.release()`。
- [x] **4. Storage Cleanup Worker 機制（P1-1）**：第 4.4 節補齊 Claim, Renewal, Completion, Failure, Reaper 5 大完整 SQL；第 13.2 節 DDL 正式新增 `worker_id` 與 `lease_timeout_at` 欄位（T56 同步更新）。
- [x] **5. Parser 記憶體行為分類（P1-2）**：第 5.3 節客觀劃分真串流、分批釋放、DOM 全載入與暫存檔處理，不作虛假固定記憶體承諾。
- [x] **6. XLSX 大檔處理策略（P1-3）**：第 7.2 節揭露 DOM 展開記憶體膨脹風險，明訂 SAX 串流解析替代方案與 2 萬列/50 萬格熔斷門檻。
- [x] **7. CSV 多階段強健偵測（P1-4）**：第 7.3 節升級為 64KB 樣本嗅探、RFC 4180 方差演算法，並於 API 規格開放手動指定編碼與分隔符。
- [x] **8. SSRF 深度防禦（P1-5）**：第 12 章補齊全部特殊 IPv4/IPv6 網段，規範 Socket 連線時之 `remoteAddress` 二次校驗與 IP Pinning。
- [x] **9. Source Location 跨欄位校驗（P1-6）**：第 6 章明訂左上角座標慣例，提供 `validateSourceLocation` 實作函式檢驗 $x2 \ge x1$ 與行號範圍。
- [x] **10. Search Vector 零停機切換（P1-7）**：第 14.2 節明訂影子欄位重建、完整性驗證、原子交換與回復流程，消除混合搜尋污染。
- [x] **11. 備份與災難復原演練（P1-8）**：第 4.3 節補齊 AES-256-GCM 雙金鑰輪替、GFS 保留政策、雙向一致性校驗器與 6 步還原應變程序。
- [x] **12. 測試矩陣狀態真實性**：完整保留 T01~T56，全數 56 項測試狀態客觀標註為 `NOT EXECUTED`，無任何虛假通過。
- [x] **13. 架構純粹性**：純 JS、無 ORM、無外部佇列、無向量庫、單一 Render 實例架構完全保留。
- [x] **14. 程式碼隔離性**：工作區未產生任何應用程式原始碼，專案純淨無污染。

---

20. 最終審查結論與 Phase 2 准入判定

- **工程規格審查結論**：**【APPROVED（准予進入 Phase 2）】**。
- **判定依據**：

1. **所有 P0 問題均已徹底解決**：

- P0-1：Failure SQL 租約逾期校驗閉環，過期 Worker 寫入被拒絕。
- P0-2：Reupload 全面回歸 Fencing Token 與非同步 Worker Claim 架構，競態與補償完備。
- P0-3：提供完整生產級 Node.js `pg` 連線池交易契約，MVCC 回滾與 rowCount 檢驗健全。

1. **所有 P1 問題均已具體落實**：

- Cleanup Worker、記憶體分類、XLSX SAX 替代、CSV 雙演算法與手動選項、SSRF Socket 校驗、座標原點與跨欄位驗證、Search Vector 原子切換、備份金鑰輪替與還原程序均已在相應章節完成定義並相互同步。

1. **全文無跨章節矛盾**：DDL、SQL、API、流程圖與測試案例高度一致。
2. **恪守工程誠實性**：56 項測試案例全數真實標記為待執行的 `NOT EXECUTED`。
3. **專案具備展開資料庫遷移腳本撰寫與目錄初始化（Phase 2）之充要條件！**